

DRISHYAM

Execution Plan

SIH26189 — step by step, per item
Ministry of Home Affairs · NCRB Women Safety Division

Generated 09 September 2026

The companion to the build roadmap. That document says what will be built and why; this one says how each item is done, which files it touches, what holds it in place afterwards, and what has to be true before it can be called finished. Read the roadmap first for the reasoning; work from this one.

How to work from this

Every item carries four lines. **files** names what changes. **tests** names what must be written alongside it — not afterwards, and not optional: every behaviour here is one somebody will later assume is still true. **done when** is a statement that can be checked on the benchmark case rather than a feeling of completeness.

Two standing rules apply to every item without being repeated. Nothing sends case content to a hosted model. Nothing states intent, identity or culpability — the system reports what a source states and marks anything it could not establish as unestablished.

Phase A — The investigator can understand what they are looking at

A1 · Capture the stated role [0.5 day]

1. Name the role group in the two patterns that already match it. `PERSON_LABEL_PATTERN` and `PERSON_INLINE_ROLE_PATTERN` both wrap the role in a non-capturing group today, so the match is made and discarded.
2. Add a readings map that reduces the label to the word a case turns on: complainant and informant to complainant; accused and suspect to accused; victim and deceased to victim; witness, driver, owner as themselves. **“Name:” is not a case role** — it becomes *named* — and “S/o Mohan Lal” names a parent, which becomes *relative*, not a role in the case.
3. Label the weaker basis honestly: “identifying himself as” becomes *self-identified*. That is how a name enters a surveillance note and it is not the same as a report assigning a role.
4. Add `find_person_roles(text)` returning name to role. Where one passage gives two readings the stronger wins — an assigned role outranks a self-given name.
5. Record the roles in extraction with the same provenance the names already carry, and store them: a `person_roles` column on the record, and `stated_role` on the entity occurrence. The role belongs to the place it was read, never to the person: the same individual can be a witness in one file and a suspect in another.

files `patterns.py` · `extraction.py` · `schema.py` · `models/entities.py` · `entity_resolution.py` · one migration

tests role read from an FIR header and from narrative prose · “Name:” yields no case role · S/o names the parent as a relative · self-identified is labelled as such · a name with no stated role gets none

done when the benchmark case reads Suresh Yadav as accused, Priya Sharma as complainant, and Suresh Yadava as self-identified — each with the line it came from

A2 · Entity summary card [1 day, after A1]

1. A service that assembles four sentences from stored rows: the role as stated and where; how many files it was found in; what it connects to and on whose evidence; and what it does **not** connect to in this case.
2. Every sentence carries its citation. No sentence is generated by a model — the facts are structured, so a template-built summary is quotable line by line where a written one is not.
3. Endpoint under the existing grounded routes, authorised the same way.
4. Surface it on the network page: clicking a node opens the summary above the source panel that already exists.
5. Carry the standing caveat, and state absence explicitly rather than leaving a gap.

files `services/entity_summary.py` (new) · `api/grounded.py` · `NetworkIntelligence.tsx` · new frontend component

tests every sentence carries a source · no sentence asserts guilt · an entity with no relationships says so plainly · an entity with no stated role says the role was not stated

done when clicking Ravi Kumar on the network answers who he is, where he came from and what he is connected to, without leaving the page

A3 · Confidence as a sentence [0.5 day]

1. Map the stored basis and confidence onto a phrase an officer can weigh: “stated directly in a call record”, “read from a screenshot”, “inferred from context”.
2. Show the phrase everywhere the number is shown today; keep the number on hover so the record is unchanged.
3. Leave the report's numeric columns alone — a court document should carry the figure.

files frontend shared formatter · NetworkIntelligence.tsx · EvidenceSourceViewer.tsx

tests each basis maps to exactly one phrase · the numeric value remains reachable

done when no bare 0.92 appears in the workspace without words beside it

A4 · Every page opens with the work [1 day]

1. One line at the top of each view stating what to do next, derived from what is already on the page.
2. Network: which links to verify first. Alerts: which cluster to read first. Review: what is unreviewed. Evidence: what failed or is unprocessed.
3. Where there is nothing to do, say that — an empty instruction line is worse than an honest one.

files each workspace view · one shared component

tests the line changes when the underlying data changes · it never states a conclusion about a person

done when each view answers “what do I do here” before it shows a table

Phase B — The questions an investigator actually asks

B1 · Entity profile page [2 days, after A1]

1. A page per entity: every way the identity was written and which file wrote it that way; every occurrence with its exact place; every relationship with the record stating it; that entity's own timeline; and its appearances in other cases where the viewer is authorised to know.
2. Reuse what exists — the source panel, the relation drawer, the summary from A2 — rather than building new surfaces for the same data.
3. Reachable from the network node, the observation table, the findings list and search.
4. An alias is a lead, not a merge: the page states the different spellings and never quietly joins them.

files `api/grounded.py` (entity routes) · `services/entity_profile.py` (new) · new workspace view · routing

tests authorisation scoped to the case · aliases listed without merging · every row opens at its source · an entity with one occurrence renders correctly

done when a name can be clicked from anywhere in the workspace and answers itself

B2 · Incident date at case creation [2 hours]

1. Add the incident window to the case intake form, writing the fields the case record already has.
2. Allow it to be set later from case settings for cases already open.
3. Where it is absent, keep saying so — the temporal sections already word this correctly.

files case intake form · cases API · settings view

tests a case created with a window produces before/during/after counts · one created without still renders

done when the benchmark case reports contact placed around the incident instead of “no incident window declared”

B3 · Cross-case identity check [2 days, after E3]

1. Given an identity in this case, report which other cases hold the same identity — the case reference and the officer to contact, never the other case's content.
2. Match on the canonical value the resolver already produces, so a number written four ways still matches.
3. Show it on the entity profile as its own section, with the disclosure limit stated on screen.
4. Audit every check: asking whether an identifier appears elsewhere is itself an access event.

files `services/cross_case.py` · entity profile page · audit

tests no field of another case leaks into the response · an unauthorised viewer gets nothing · the check is audited

done when “this number appears in two other cases” is answerable without either officer seeing the other's file

Phase C — Alerts that tell a story

C1 · Narrative pattern alerts [2 days]

1. Assemble the sourced facts behind a pattern in time order and render them as a sequence: time, what the source states, and the file and place it is written.
2. Close every sequence with the same line — the sequence is recorded, the meaning is the reader's.
3. State the gaps inside the sequence too: “between these, no recorded contact” is part of the story.
4. No sentence may contain a because, a motive, or a relationship the source did not state.

files alerts/network_rules.py · alert rendering in the workspace · report alert section

tests every line carries a file and a place · no rendered sequence contains a causal clause · a pattern with one fact still renders

done when an alert reads as a story an officer can act on and states nothing about why

C2 · Five new pattern rules [2 days]

1. **Relay** — A contacts B and B contacts C inside a short window.
2. **Converging location** — several distinct entities placed at one location inside a window.
3. **Late-arriving identity** — an identity that first appears well after the case opened.
4. **Vehicle corridor** — one vehicle recorded at several places in time order.
5. **Sudden silence** — a pair in regular recorded contact that stops. Nothing reports this today and it is often the strongest signal in a case.
6. Each rule states what it observed and what it does not establish, and is idempotent on reprocessing.

files alerts/network_rules.py · temporal.py

tests each rule fires on the benchmark case where expected and stays silent where not · reprocessing raises no duplicates · no rule discloses another case

done when five new patterns appear on the benchmark case, each opening at its evidence

C3 · Alerts page states the work [0.5 day, after C1]

1. A line above the list: how many patterns, which cluster in time, and which links to verify first.
2. Derive it from the bridge relationships and the temporal data already computed.

files alerts view

tests the line reflects the current filters · it names links rather than people

done when the page opens with an instruction, not a count

Phase D — The report as a working surface

D1 · In-app report viewer with search [2 days]

1. Render report pages through the endpoint that already renders evidence pages, and page through them in a panel.
2. Search inside the document. The renderer returns the rectangle of every match, so results are marked in place rather than counted.
3. Reuse the mark styling from the evidence viewer so a highlight means the same thing everywhere.

files api/reports (page render) · new report viewer component · reuse of the source panel styling

tests search marks every occurrence and reports the count honestly · a report that fails to render says so
· access is authorised and audited

done when a nineteen-page report can be read and searched without downloading it

D2 · A finding opens its evidence [0.5 day, after D1]

1. Findings already carry a number, a file and an exact place. Make each one a control that opens the existing source panel at that place.
2. Do the same for the source crops already printed in the report.

files report viewer · EvidenceSourceViewer.tsx

tests every numbered finding resolves to a source · a finding whose evidence is gone says so rather than failing

done when F-07 in the report opens the row it was read from

D3 · Report questions, answered offline [1 day, after D1]

1. Questions of fact — where did this come from, what supports it, what connects these two — go to the existing case assistant, which has no model behind it.
2. Questions of wording surface the explanation the report already stores rather than writing a new one.
3. Anything else is declined with a plain sentence saying why, and the scope is stated on screen.

files report viewer · services/case_assistant.py

tests no report content reaches a hosted provider · a question outside scope is declined rather than guessed

done when a reader can ask about a finding and get an answer with its source, offline

Phase E — Integrity, and where blockchain genuinely belongs

E1 · Chain verification [1 day]

1. Walk the audit chain for a case from its first entry, recompute each hash from the stored payload, and confirm each entry carries the previous one's hash.
2. Report the count and the outcome — intact, or the exact entry where it breaks and what differs.
3. Surface it three ways: an endpoint, a badge in the workspace, and a line in the court annexure.
4. Today trustify/verify checks the report and manifest hashes — that part works — and only echoes the chain head. A chain nobody walks is a table that looks like tamper-evidence.

files services/trustify.py · api · integrity view · reporting.py

tests an intact chain verifies · a tampered entry is located exactly · a case with no entries reports that honestly

done when “prove the chain holds” has an answer, on screen and in the annexure

E2 · Merkle root per case [1 day, after E1]

1. Build a Merkle tree over the evidence hashes of a case; print the root in the report and store it with the receipt.
2. Provide an inclusion proof for any single file: the short path proving that file was in the set, without revealing the others.
3. State plainly what the root proves and what it does not — it fixes the set at a moment, it does not authenticate the contents.

files services/trustify.py · reporting.py · api

tests a proof verifies against the root · a file not in the set fails · adding a file changes the root

done when a court can be shown that one file was in the case at report time without seeing the rest

E3 · Shared hash ledger for cross-case matching [3 days, after E1]

1. Publish only sha256(canonical identifier) with a case reference and a contact — never the identifier, never any case content. A hash cannot be reversed and discloses nothing on its own.
2. Append-only and chained, so an entry cannot be removed or backdated by whoever holds the store.
3. Off by default, with the same double gate the external-model switch uses, and every publish audited.
4. This is the one place in this product where the parties do not trust each other, which is the only place a shared ledger earns its cost. Everywhere else a local hash chain is the honest answer.

files services/ledger.py (new) · settings gate · audit · cross_case.py

tests nothing but a hash and a reference leaves · the gate defaults closed · a publish is audited · matching works across differently written forms of the same identifier

done when two districts can discover a shared identifier without either seeing the other's case

Phase F — Fewer places to look

F1 · Seventeen views to eight [2 days]

1. Entities & Graph and Network Intelligence become one **Network**. Corroboration, Contradictions and Review Queue become one **Review**. Integrity, Custody/Audit and Processing become one **Integrity**. Reports and Report Versions become one **Reports**.
2. Nothing is deleted. Each merged view keeps the sections it absorbed, as tabs or as sections within the page.
3. Update the sidebar, the routing and any deep links that name the old views.

files Workspace.tsx · sidebar · routing · view components

tests every feature reachable before is reachable after · no route 404s · deep links still resolve

done when the sidebar holds eight destinations and the demo can show all of them

Phase G — Finishing the workflow

G1 · Notes on entities and relations [1 day, after B1]

1. A note belongs to a case object, an author and a time, and is never mixed into extracted facts.
2. Show notes on the entity profile and the relation drawer; include them in the case file report, marked as investigator commentary rather than evidence.

files models · api · entity profile · relation drawer · reporting.py

tests a note is attributed and timestamped · notes never appear as findings · deleting is recorded, not silent

done when an investigator can write down what they know beside what the system read

G2 · Export [1 day]

1. CSV and JSON for entities, relationships and numbered findings, carrying the source columns.
2. Exporting is a disclosure: audit it, and apply the same protected-identity rule the reports use.

files api · workspace export controls

tests the export carries the source of every row · protection is applied · the action is audited

done when a case can be handed to another system without anybody retyping it

G3 · Requisition draft [1 day, after C2]

1. From the network and the temporal data, draft what to request next: which numbers, which date ranges, and why each is being asked for.
2. Produce it as text the officer edits and sends. The system drafts; it does not submit.

files services/requisition.py (new) · workspace

tests every number in a draft traces to a stated relationship · the draft states its own basis

done when the page that shows the network can produce the next request from it

G4 · What changed since you were last here [1 day]

1. Record when each user last opened a case; summarise what arrived since — new evidence, new relationships, new patterns, new review decisions.
2. Show it on opening the case, dismissible, never blocking.

files models · api · overview view

tests the digest is per user · it reports nothing the user cannot access · an unchanged case says so

done when opening a case answers “what is new” before anything else

Order of work

Step	Items	Effort	Why here
1	A1 + A2	1.5 d	Half a day unblocks four items. The investigator's first question, answered.
2	B1	2 d	The largest missing surface in the product.
3	E1	1 d	The theme is Blockchain & Cybersecurity and the chain is currently unproven.
4	B2	2 h	One field wakes an entire dormant feature.
5	C1 + C2	4 d	The largest readability gain after A2.
6	D1 + D2	2.5 d	Closes the loop from report back to evidence.
7	F1	2 d	Worth as much to the demo as any new feature.
8	E3	3 d	The blockchain story, told where it is true.
9	A3, A4, C3, D3, E2, G1–G4	7 d	Polish, in whatever order time allows.

About twenty-five working days for one person. Phases C, D and F depend on nothing in Phase B, so three people can run them in parallel once step 2 is done.

Rules that apply to every item

Rule	Why
No case content reaches a hosted model.	It is the strongest claim this product makes and one careless feature ends it. Where a summary is needed, the facts are structured and a template-built sentence is citable where a written one is not.
Nothing states intent, identity or culpability.	A confident wrong lead costs an investigation more than a missing one. The system reports what a source states.
Every new statement carries its source.	A relationship with no source cannot be written at all; the same standard applies to anything added here.
Absence is reported, never left blank.	A silent gap reads as “nothing there”. Say that nothing was recorded, and that this is not the same as nothing having happened.
Tests are written with the item, not after it.	Every behaviour in this plan is one somebody will later assume is still true.

Nothing in this plan claims a capability is already implemented. Where an item says a thing does not work today, that was verified in the running system.